Anchored Counterfactual Attribution of Model Degradation to ETL Pipeline Operators

Laiba Mazhar
Data Scientist
Lahore, Pakistan
lm4442172@gmail.com

Maryam Sarfraz
Computer Scientist
Lahore, Pakistan
sarfrazmaryam123@gmail.com

Abstract*—Production machine learning models are fed by ETL pipelines of tens to hundreds of operators. When model quality drops between two scheduled runs with no code deployed, engineers must determine which operator is responsible. Current practice walks the pipeline in topological order, swapping in the new run's data one stage at a time. We show this is the marginal-contribution vector of a single permutation and is therefore order-dependent: on our benchmark the *sign* of attributed blame reverses between two valid topological orders of the same DAG. We formulate the problem as a cooperative game. Holding the pipeline fixed, each operator executes in either of two states—as in a known-good *reference* run or as in the degraded *incident* run—and a *hybrid replay* executes the DAG under an arbitrary state assignment. The Shapley value of the resulting game decomposes the observed degradation exactly. Our central finding is negative and then constructive: the Shapley value is correct as accounting but poor as triage, ranking the true culprit first in only 60% of measurable incidents, and failing precisely when the injected fault does less damage than co-occurring benign drift. Leave-one-out ablation ranks well on synthetic data but its attributions deviate from the observed degradation by up to 0.246 AUC. We introduce the *incident-anchored Shapley value*, which holds a designated set of exogenous operators at incident state and takes the Shapley value of the resulting sub-game. It recovers standard Shapley and leave-one-out as boundary cases, attains 100% top-1 accuracy in every severity regime, and retains exact decomposition to within 3×10^{-17}. On real NYC taxi partitions the separation sharpens: comparing a Monday against a Saturday, leave-one-out collapses to 0.50 while the anchored value holds at 1.00. Output-hash pruning and prefix memoisation reduce 2^{33} hybrid replays to three model fits on a 33-operator DAG.

***Index Terms*—data pipelines, model degradation, root cause analysis, Shapley value, data-centric AI, ETL**

I. Introduction

A model scores 0.87 AUC on Monday. On Tuesday the same Airflow DAG runs, nothing was deployed, and it scores 0.81. Which task caused it?

This is among the most common failure modes in production machine learning, and existing tooling addresses it poorly. Monitoring systems [13] report *that* a distribution moved; in a deep DAG a single upstream change makes every downstream node move, so a monitor fires everywhere and ranks nothing. Lineage systems [5] report what is upstream of what, narrowing the suspects to "everything upstream." Pipeline debuggers—MLINSPECT [1], [2] is the strongest—instrument a pipeline into a dataflow DAG and propagate annotations to detect data-distribution bugs *within one execution*; they do not compare executions across time and do not assign quantitative blame.

So the work is done by hand, and the manual procedure has a defect nobody has named: it depends on the order the engineer happens to walk the DAG in.

A. Contributions

1) **Formulation** (Sec. III). Temporal hybrid replay of a fixed pipeline over two snapshots as a cooperative game whose Shapley value decomposes observed degradation exactly. Verified numerically to 6×10^{-17}.
2) **A negative result** (Sec. VI-B). The Shapley value is the wrong estimator for triage, failing when the fault is smaller than co-occurring benign drift.
3) **The anchored Shapley value** (Sec. IV). A one-parameter family spanning standard Shapley and leave-one-out, attaining 100% top-1 accuracy while preserving exact decomposition.
4) **Tractability** (Sec. V). Output-hash pruning and prefix memoisation, with the collapse regime identified.
5) **A fault-injection benchmark** over synthetic and real NYC taxi pipelines, with ground-truth culprits, monotone severity ladders, and *benign controls*—faults that violate data-quality constraints loudly while causing no model damage.
6) **A real-data result** (Sec. VI-G) showing that leave-one-out, indistinguishable from the anchored value on synthetic data, fails under real weekday/weekend drift.

We do not have a production deployment, we have not integrated with an orchestrator, our real-data evaluation is one dataset and one pipeline topology, and the anchored estimator's axiomatic characterisation is stated but not proven. Section VIII is explicit about all of this.

II. Problem Statement

A pipeline is a DAG $G = (V, E)$ of operators, each computing $d_i = f_i(\text{inputs}, \text{params}_i)$. Two executions are given: a *reference* run R (last known-good) and an *incident* run I (degraded). Between them each operator may differ in its source partition, its configuration, or its code. We encode all three uniformly as an operator *state* $\sigma_i \in \{R, I\}$.

TABLE I
BLAME UNDER ALL TOPOLOGICAL ORDERS (7-OPERATOR PIPELINE)

Scenario	Orders	Stagewise top-blamed	Anchored
unit change	6	`agg_transactions` (6/6)	correct
predicate flip	6	`filter_active` (6/6)	correct
schema drift	6	*a source* (6/6, wrong)	correct
unit + predicate	6	`agg_transactions` (5/6)	correct

A downstream model is trained on the sink dataset and evaluated on a fixed probe set. Writing u for that utility, the observed degradation is $\Delta = u(V) - u(\emptyset)$.

The attribution problem. Distribute Δ across operators such that the assignment identifies which operators to repair.

III. Hybrid Replay and the Degradation Game

Definition 1 (Hybrid replay): For $S \subseteq V$, $\Pi(S)$ executes the DAG with every operator in S in incident state and every operator in $V \setminus S$ in reference state:

$$d_i(S) = f_i\Big(\{d_j(S) : (o_j, o_i) \in E\},\ \text{params}_i^{\sigma_i(S)}\Big) \quad (1)$$

where $\sigma_i(S) = I$ if $o_i \in S$ and R otherwise.

$\Pi(\emptyset)$ reproduces the good run, $\Pi(V)$ the bad one, and the $2^n - 2$ hybrids between them interpolate.

Definition 2 (Degradation game): With $u(S)$ the model utility on $\Pi(S)$'s output, $v(S) = u(S) - u(\emptyset)$.

Then $v(\emptyset) = 0$ and $v(V) = \Delta$. The probe set is held fixed across every coalition; otherwise $u(S)$ compares different things and every property below is vacuous. The Shapley value

$$\phi_i = \sum_{S \subseteq V \setminus \{i\}} \frac{|S|!\,(n - |S| - 1)!}{n!} [v(S \cup \{i\}) - v(S)] \quad (2)$$

satisfies *efficiency*: $\sum_i \phi_i = \Delta$. The degradation is *decomposed*, not merely scored. No baseline we are aware of has this property.

A. Current Practice is a Single Permutation

Walking the DAG in topological order and swapping stages in one at a time produces, for order π, the vector $m_i^\pi = v(S_\pi(i) \cup \{i\}) - v(S_\pi(i))$ where $S_\pi(i)$ precedes i. This is the marginal-contribution vector of *one* permutation; (2) averages over all $n!$.

Proposition 1: Single-permutation stagewise diagnosis violates the symmetry axiom and is order-dependent.

The empirical form is stark. Enumerating *every* valid topological order of the seven-operator pipeline over four fault scenarios, **8 of 28 operator/scenario pairs receive blame of both signs** depending only on the order chosen (Table I). For `load_customers` the assigned blame ranges from -0.0556 to $+0.0405$: a spread of 0.096 AUC on incidents whose total magnitude is 0.03–0.11 AUC. Two engineers running the identical manual procedure on the identical incident reach opposite conclusions about whether an operator helped or harmed.

Worse than noisy: on the schema-drift scenario stagewise diagnosis blames an innocent *source* operator in all six orders, never the culprit. The anchored value is order-independent by construction and is correct in all four (Fig. 1).

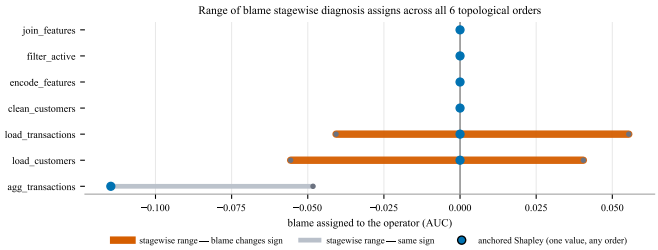

Fig. 1. Range of blame stagewise diagnosis assigns to each operator across all six topological orders of the unit-change scenario. Bars crossing zero change sign with the order alone. The anchored value is a single point regardless of order.

IV. The Incident-Anchored Shapley Value

Section VI-B shows the Shapley value ranks culprits worse than naive leave-one-out. The reason is a conditioning difference. Leave-one-out evaluates $v(V) - v(V \setminus \{i\})$, the marginal effect *at the actual incident configuration*, where benign drift is held fixed and cancels. Shapley averages over all 2^n configurations, including many where the benign drift has not happened, so drift receives a real, nonzero share. Both are right about different questions, and the space between them is empty in the literature.

Definition 3 (Incident-anchored Shapley value): For an anchor set $B \subseteq V$, let $v_B(T) = v(T \cup B) - v(B)$ be the sub-game on $V \setminus B$. The anchored value ϕ^B is the Shapley value of v_B, with $\phi_i^B = 0$ for $i \in B$ and, writing $m = |V \setminus B|$,

$$\phi_i^B = \sum_{S \subseteq V \setminus (B \cup \{i\})} \frac{|S|!\,(m - |S| - 1)!}{m!} [v(S \cup B \cup \{i\}) - v(S \cup B)] \quad (3)$$

Theorem 1 (Boundary cases): $B = \emptyset$ gives the standard Shapley value. $B = V \setminus \{i\}$ gives $\phi_i^B = v(V) - v(V \setminus \{i\})$, the leave-one-out value of i.

Both are verified as assertions to machine precision before any result is reported.

Theorem 2 (Anchored efficiency): $\sum_{i \notin B} \phi_i^B = v(V) - v(B)$.

The anchored values decompose exactly the part of the degradation not already explained by the anchor. Measured maximum deviation across all 29 synthetic configurations: 2.78×10^{-17}.

A. Choosing the Anchor

The principled default anchors the *exogenous* operators—sources, with no parents. Their state differs because the world moved, not because the pipeline did. One cannot repair the fact that Tuesday's data is not Monday's, so charging blame to it is accurate accounting and useless triage. This is a modelling decision and we present it as one.

V. Computing It

Each $v(S)$ costs a pipeline replay plus a model fit; 2^n of them is hopeless. Three mechanisms make it practical.

Ancestral sufficiency. $u(S)$ depends only on the sink, so operators outside $\mathrm{Anc}^*(\text{sink})$ are null players. In real Airflow DAGs most tasks are sensors, notifications and cleanup; this removes them for free.

Output-stability pruning. If operator i's output is byte-identical under both states given the same inputs, all downstream computation is identical and $v(S \cup \{i\}) = v(S)$. This is detectable by *hashing*, with no model fit—one hash replaces one replay-and-retrain. The operators surviving this test form the *active frontier* F.

Prefix memoisation. Hybrid replays share large prefixes. We cache operator outputs on $(\text{operator}, \text{state}, \text{hash(inputs)})$ and utilities on the sink hash, so distinct coalitions producing an identical sink share a single model fit.

Beyond $n \approx 14$ we use permutation sampling over the free players with the anchor always included.

VI. Evaluation

A. Setup

Synthetic pipeline. A seven-operator churn feature build: two sources, a cleaning step, an aggregation, a join, a filter, an encoder; 4000 customers per day. $n = 7$ keeps the 128-coalition lattice brute-forceable so Monte-Carlo estimates can be checked against exact ground truth.

Benign drift. Reference and incident runs read *different days* of source data from the same distribution, so every scenario layers a real fault on natural churn and every downstream node's output changes whether or not it is to blame. This is what makes per-node monitoring fail, and it is what breaks the plain Shapley value.

Real pipeline. Fig. 2 shows the nine-operator NYC taxi pipeline used in Sec. VI-G, with the two operators carrying injected faults highlighted. CULPA is not told which they are.

Faults. Five types with severity ladders (29 configurations): unit change, schema drift, filter-predicate flip, MNAR null spike, biased join fan-out. Two *benign controls*—uniform join fan-out (35% duplicate keys) and MCAR nullness (55% of rows dropped)—violate data-quality constraints loudly while causing no model damage.

Fig. 3 plots what each fault actually costs as severity rises, and it is why the evaluation reports exclusions. Several configurations sit below any measurable effect, and one ladder is not even monotone: for schema drift *which* column is dropped matters more than how many, so losing `tenure` costs an order of magnitude more than losing `total_amount`. An injected fault is a hypothesis about damage and has to be checked before attribution is scored against it.

Baselines. Per-node distributional drift, single-permutation stagewise swap (current practice), and leave-one-out.

B. The Negative Result

On the initial gate experiment (10 scenarios), leave-one-out beat the Shapley value, as shown in Table II.

TABLE II
Initial gate experiment, 8 harmful scenarios

Method	p@1	MRR	Blame mass
Leave-one-out	**1.00**	**1.00**	**0.87**
Per-node drift	0.88	0.94	0.51
Shapley (exact)	0.75	0.88	0.62
Stagewise	0.38	0.69	0.41

TABLE III
Only the anchored value decomposes the degradation

	Max deviation	Mean
Anchored, $\lvert\sum\phi - (v(V) - v(B))\rvert$	$\mathbf{2.78 \times 10^{-17}}$	—
Leave-one-out, $\lvert\sum\phi - v(V)\rvert$	2.46×10^{-1}	9.6×10^{-2}

Shapley lost exactly the two scenarios where the fault did *less* damage than benign drift: biased join fan-out ($\Delta = -0.0016$) and MNAR null spike ($\Delta = -0.0028$), against a per-source benign drift magnitude of 0.0556. In both it ranked a source above the culprit—which, as accounting, is correct. This motivated the anchored value.

C. Accuracy Against the Fault-to-Drift Ratio

Let $r = |v(\{\text{culprit}\})| / \max_s |v(\{s\})|$ over sources s. Figure 4 plots precision@1 against r for both pipelines.

Across the 15 measurable synthetic configurations the anchored value attains p@1 $= 1.00$, against 0.60 for plain Shapley, 0.87 for per-node drift and 0.33 for stagewise. The predicted mechanism is confirmed: plain Shapley degrades to 0.20 when the fault is small relative to benign drift and recovers to 1.00 when it dominates, while the anchored value is flat because anchoring removes benign drift by construction.

D. Anchored Value versus Leave-One-Out

On synthetic data the two rank identically—15/15 agreement on p@1. They differ in what the numbers mean (Table III). Leave-one-out's attributions miss the observed degradation by up to 0.246 AUC, larger than most of the incidents being diagnosed: it double-counts interacting faults, and in one scenario reported that an operator *helped* by $+0.04$ when its true anchored contribution was $+0.005$.

E. A Worked Incident

Fig. 5 shows the tool's output on a compound incident in the real taxi pipeline of Fig. 2: a filter predicate silently flips and, independently, three feature columns vanish after an upstream rename. The run loses 0.0379 AUC. Anchoring the two sources removes the benign weekday/weekend drift, leaving 0.0316 to attribute, which the anchored value splits -0.0171 to `filter_valid` and -0.0145 to `encode` with a residual of 2×10^{-17}. The remaining seven operators receive exactly zero, having been eliminated by output-hash pruning without a single model fit.

That split is the operational point. A method that only ranks says `filter_valid` is worse; the decomposition says fixing

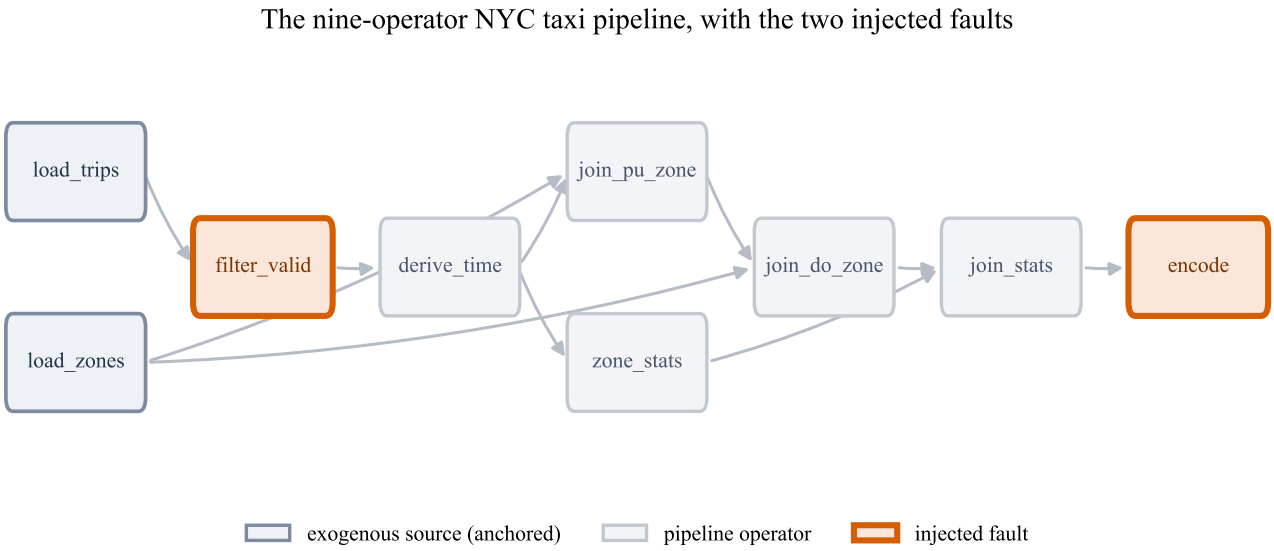

Fig. 2. The nine-operator NYC yellow taxi pipeline. Sources (left) are exogenous—their state differs because the world moved between the two runs, not because the pipeline did—so they form the default anchor set. The two highlighted operators carry the injected faults of Fig. 5.

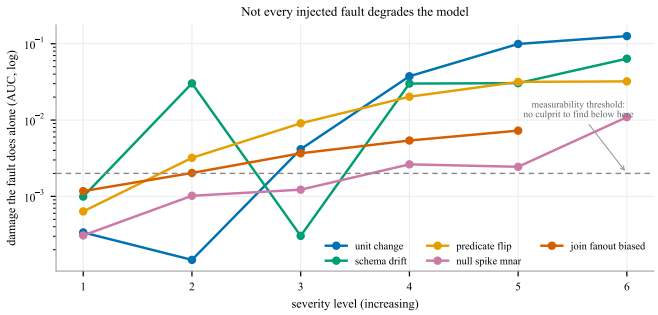

Fig. 3. Damage each fault causes in isolation, by severity level. Below the dashed threshold there is no degradation to attribute and no culprit to find. Schema drift is non-monotone: the identity of the dropped column dominates its count.

it alone recovers just over half the loss, which is what decides whether one repair is enough.

F. Benign Controls

Table IV contrasts the benign controls with a genuinely harmful fault. Uniform fan-out and MCAR nullness fire every data-quality constraint a monitor has—35% duplicate keys, 55% of rows dropped—and cost nothing. The method assigns them near-zero blame. **Not every data-quality violation is an incident**, and a constraint checker cannot tell the two apart because it never looks at the model.

G. Real Data: NYC Yellow Taxi

A nine-operator pipeline over NYC TLC yellow taxi records for January 2024: two sources, a validity filter, time-feature derivation, a per-zone aggregate, two zone joins, a stats join, and an encoder. Task: predict a generous tip (tip/fare > 0.20) for credit-card trips; 60,000 trips per partition. `tip_amount` and `total_amount` are dropped before modelling—the latter includes the tip and would leak the label outright.

Reference and incident runs read two different real days. Table V gives two regimes.

Leave-one-out fails on real data. On the synthetic workload it scored 1.00 and was separable from the anchored value only on the efficiency property. Here, wherever the fault is smaller than the benign weekday/weekend shift, it drops to 0.00 alongside plain Shapley and stagewise, and the anchored value alone holds at 1.00 (Fig. 4b). Real drift is large and asymmetric in a way our generator's was not: Saturday differs from Monday in trip distance, hour distribution and tipping behaviour simultaneously, so removing a source from the full

TABLE IV
CONSTRAINT VIOLATION IS NOT MODEL DAMAGE

Injected fault	Constraint	Δ (AUC)	max $\lvert\phi\rvert$
Uniform join fan-out, 35%	fires	+0.0001	0.0078
MCAR nullness, 55% rows	fires	+0.0006	0.0085
Silent unit change	silent	−0.1149	0.0996

TABLE V
REAL NYC TAXI, PRECISION@1 OVER VALID TEST CASES

	Mon 8 vs Tue 9	Mon 8 vs Sat 13
Benign drift max $\lvert v(s)\rvert$	0.0104	0.0063
Valid test cases	6 / 19	8 / 19
Anchored Shapley	**1.00**	**1.00**
Per-node drift	0.50	0.62
Shapley	1.00	**0.50**
Leave-one-out	1.00	**0.50**
Stagewise	1.00	**0.50**

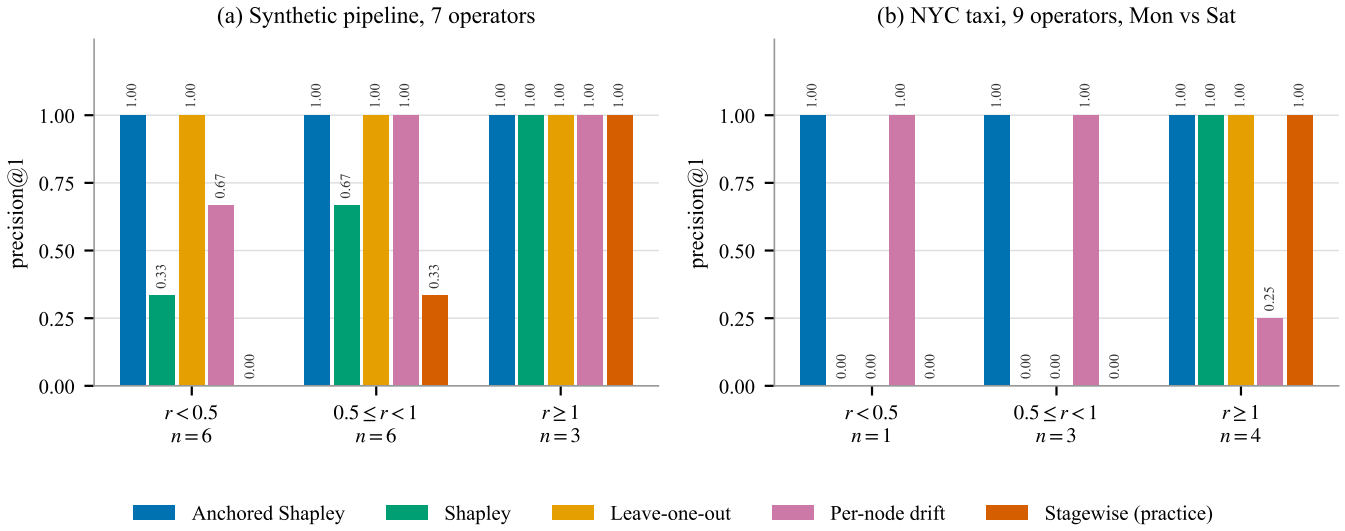

Fig. 4. Precision@1 by fault-to-drift ratio r. The anchored value is flat at 1.00 in every regime. On the synthetic pipeline (a), plain Shapley degrades below $r = 1$ while leave-one-out does not. On real NYC taxi partitions (b), leave-one-out collapses alongside Shapley and current practice, and the anchored value is the only estimator that survives.

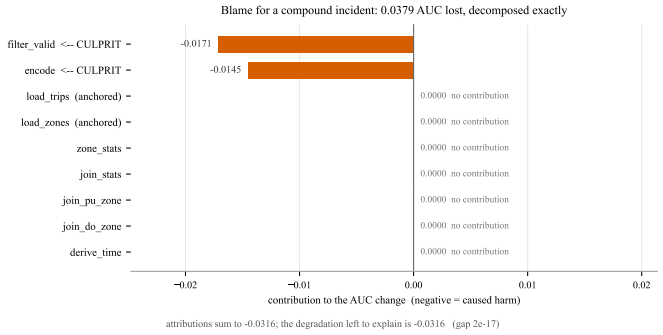

Fig. 5. Blame assigned on a compound incident in the taxi pipeline. Both culprits are identified and the attributions sum to the degradation being explained.

coalition—exactly what leave-one-out does—moves the utility more than the injected fault does. Anchoring removes that term by construction instead of competing with it.

Exclusions. Only 6–8 of 19 configurations are valid test cases. Two rules apply, both stated before results were read. First, *the injected fault causes no damage in isolation* (11 configurations): all four `broken_join_key` levels and all three `join_fanout` levels. Duplicating rows in the zone lookup is textbook join fan-out, but a left join on `DOLocationID` reproduces the same borough value, so it reweights the training set uniformly and costs nothing; scoring a method for failing to blame an operator that caused no harm would penalise it for being right. Second, *the fault is harmful but benign drift offset it* (2 configurations). The second rule is itself a finding: on real data a genuine fault can be masked entirely by a favourable partition, and a monitor watching only end-to-end model quality would not fire at all.

TABLE VI
COST AS DAG WIDTH GROWS

n	2^n	Fits	Cache hit	$\lvert F\rvert$	MC time
6	64	4	91.3%	2	<0.01 s
9	512	8	98.5%	3	<0.01 s
12	4,096	16	99.7%	4	<0.01 s
15	32,768	3	99.4%	5	0.19 s
21	2.1×10^6	3	99.7%	7	0.41 s
27	1.3×10^8	3	99.8%	9	0.68 s
33	8.6×10^9	3	99.8%	11	0.99 s

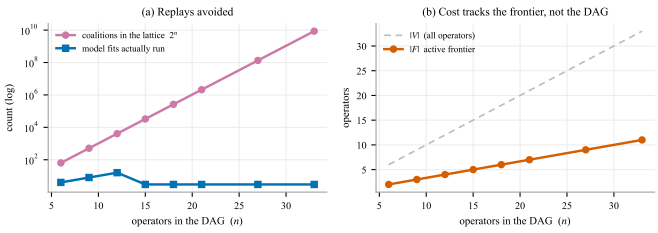

Fig. 6. (a) The coalition lattice grows as 2^n while the number of model fits stays flat. The step at $n = 15$ is an artifact of the method, not the workload: exact Shapley is computed up to $n = 14$ and evaluates more coalitions than the sampled estimator used beyond it. (b) The active frontier grows linearly and far below $|V|$, which is what the flat fit count actually reflects.

H. Scale

A generated fan-in DAG with m source branches ($n = 3m+3$) gives the cost figures in Table VI.

How to read this. The headline ratio is real but is *not* a function of n (Fig. 6). Only three distinct sink states exist in this workload, because every other operator is state-insensitive and its flip is caught by the output hash before any model is fit. Cost tracks $|F|$, not $|V|$. That identifies its own failure

mode: the reduction collapses when a change touches many operators simultaneously—a shared library upgrade, a global config change, a backfill.

VII. Related Work

A. Debugging and Orchestrating ML Pipelines

MLINSPECT [1], [2] extracts a dataflow DAG from a pipeline and propagates lineage annotations to detect data-distribution bugs. It is the closest prior art and is complementary: it inspects one execution, whereas we compare two and attribute a quantity. Its DAG extraction is what a production implementation of our method should reuse. MODYN [3] treats the pipeline itself as the managed object, orchestrating data-centric retraining over evolving datasets; it decides *when* to retrain, where we ask why the last retrain got worse. Recent position work on next-generation data engineering pipelines [4] argues that pipelines need first-class observability semantics rather than bolt-on monitoring, which is the gap this paper addresses for one specific question. Lineage systems [5] supply the topology our replay engine consumes but say nothing about magnitude.

B. Shapley Values in Data Management

SHAPLEYPIPE [6] applies Shapley values to pipeline operators for *construction*—which operator to select to maximise offline accuracy. That is a design-time AutoML question over a space of candidate pipelines; ours is a run-time forensic question about one deployed pipeline whose inputs changed. The player sets, games and answers differ. A recent survey [7] maps the broader use of Shapley values across data analytics, and asymptotic analysis of the dataset-valuation game [8] characterises its large-sample behaviour.

Efficient estimation is the shared bottleneck. SHAPLEIG [9] selects which coalitions to evaluate by Bayesian experimental design, which is directly applicable to our setting because our value function is expensive for exactly the reason theirs is—each evaluation trains a model. OPERATORSHAP [10] accelerates estimation in a different operator-valued setting. Our output-hash pruning is complementary to both: it removes evaluations rather than choosing among them, and it composes with any sampling scheme.

C. Data Valuation

Data valuation methods value *training examples*; we value *operators*. The estimation machinery transfers, the semantics do not. Recent work values data for LLM alignment via Shapley [11] and argues for influence-based auditing as a cheaper alternative [12]. The same efficiency-versus-fidelity tension appears here, and is why we report exact values wherever brute force is affordable.

D. Data Quality and Agent Benchmarks

Data-quality systems [13] verify declarative constraints, and SPADE [14] synthesises such assertions automatically for LLM pipelines. Our benign controls (Table IV) show that constraint violation and model damage are distinct properties: a checker that never looks at the model cannot separate them. Benchmarks such as ELT-Bench [15] and DATAGOVBENCH [16] evaluate whether agents can *author* or *govern* pipelines; we ask which operator in an existing pipeline broke.

E. Root Cause Analysis

Root-cause analysis for production systems has matured towards causal formulations [17] and agentic, evidence-grounded reasoning [18]. Both target service-level incidents where the signal is telemetry; our object is a dataflow DAG and our signal is model utility, which admits the exact decomposition telemetry-based methods cannot offer. A further body of recent work addresses "AI-powered data pipeline observability," predominantly in preprint repositories and low-tier venues, without formal guarantees, ground-truth benchmarks, or reproducible artifacts.

F. Cooperative Game Theory

The anchored value's restriction to feasible coalitions relates to conjunctive permission structures [19] and, when coalitions must be DAG-connected, the Myerson value [20]. We state the connection without proving a characterisation; doing so is the first open problem in Sec. IX.

VIII. Limitations

One real dataset, one pipeline shape. Whether the crossover sits at the same ratio for other domains, and for deep rather than wide pipelines, is untested.

The fault suite is weaker than intended. Eleven of nineteen taxi configurations were excluded because the injected fault does no damage, leaving the real-data conclusion resting on eight test cases.

Few valid cases at the crossover. The regime separating the anchored value from every baseline contains four configurations below $r = 1$. The separation is clean—1.00 against 0.00—but it is four points, not a curve.

The probe set must exist. The method rests on a fixed evaluation set representing ground truth. Where labels arrive with long delay, u is not computable as defined. This is the most serious practical limitation.

Deterministic utility. We use a deterministic learner with common random numbers. For stochastic learners u is a random variable, efficiency holds only in expectation, and existing Shapley concentration bounds—which assume a deterministic oracle—do not directly apply.

Anchor choice is a modelling decision. If a source change is itself the repairable fault—a misconfigured extraction job rather than genuine world drift—anchoring hides the culprit by construction. Detecting that case is unsolved.

Hashing requires determinism. Operators involving sampling, timestamps, or `ORDER BY` without a total order break output-stability pruning.

A methodological note. Three of our injected faults measured no damage for reasons unrelated to the method: a pure rescale that downstream standardisation undid, a label recomputed from the corrupted columns so the target moved

with the features, and a join fault against a feature the workload gave no signal to. All three looked like plausible faults and none was. Any fault-injection benchmark in this space must *verify* that each injected fault degrades the model before scoring attribution against it.

IX. Conclusion

Attributing model degradation to pipeline operators can be posed as a cooperative game over hybrid replays of a fixed pipeline across two temporal snapshots. The Shapley value decomposes degradation exactly but ranks poorly under co-occurring benign drift. The incident-anchored Shapley value recovers both standard Shapley and leave-one-out as boundary cases, ranks perfectly across every severity regime tested, preserves exact decomposition, and—unlike leave-one-out—survives real day-over-day drift.

Open problems, in the order we would attack them: an axiomatic characterisation of ϕ^B and its precise relation to the conjunctive permission value; automatic anchor selection, distinguishing genuine exogenous drift from a repairable extraction fault; concentration bounds under a stochastic utility oracle; the broad-change regime where the active frontier is large and pruning fails; and federated attribution across data silos without centralising raw data.

Reproducibility

All numbers are produced by the accompanying artifact; every experiment runs in under two minutes on a laptop. The boundary theorems are asserted to machine precision before any result is reported.

References

[1] S. Grafberger, S. Schelter *et al.*, "mlinspect: a data distribution debugger for machine learning pipelines," in *Proc. ACM SIGMOD*, 2021.

[2] S. Grafberger *et al.*, "Data distribution debugging in machine learning pipelines," *The VLDB Journal*, vol. 31, 2022.

[3] M. Böther *et al.*, "Modyn: data-centric machine learning pipeline orchestration," arXiv:2312.06254, Dec. 2023.

[4] K. M. Kramer *et al.*, "Towards next generation data engineering pipelines," arXiv:2507.13892, Jul. 2025.

[5] OpenLineage, "An open framework for data lineage collection and analysis." [Online]. Available: https://openlineage.io

[6] J. Chang, C. Liu, J. Huang, S. Zheng, R. Mao, and J. Qin, "ShapleyPipe: hierarchical Shapley search for data preparation pipeline construction," arXiv:2510.27168, Oct. 2025.

[7] H. Lin *et al.*, "A comprehensive study of Shapley value in data analytics," arXiv:2412.01460, Dec. 2024.

[8] M. Tamine *et al.*, "An asymptotic analysis of the Shapley value for dataset valuation," arXiv:2607.03374, Jul. 2026.

[9] D. Rundel *et al.*, "ShaplEIG: Bayesian experimental design for Shapley value estimation," arXiv:2606.02247, Jun. 2026.

[10] J. Stiller *et al.*, "OperatorSHAP: fast and accurate Shapley value estimation for neural operators," arXiv:2606.28065, Jun. 2026.

[11] M. Tamine *et al.*, "Shapley-based data valuation for LLM alignment via sequential preference optimization," arXiv:2512.15765, Dec. 2025.

[12] Y. Song *et al.*, "Beyond Shapley: an influence-based data auditing pipeline for LLM alignment and evaluation," arXiv:2607.22766, Jul. 2026.

[13] S. Schelter *et al.*, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, 2018.

[14] S. Shankar *et al.*, "SPADE: synthesizing data quality assertions for large language model pipelines," arXiv:2401.03038, Jan. 2024.

[15] "ELT-Bench: an end-to-end benchmark for evaluating AI agents on ELT pipelines," *Proc. VLDB Endowment*. doi:10.14778/3773749.3773750.

[16] Z. Liu *et al.*, "DataGovBench: benchmarking LLM agents for real-world data governance workflows," arXiv:2512.04416, Dec. 2025.

[17] S. Jha *et al.*, "Causal AI-based root cause identification: research to practice at scale," arXiv:2502.18240, Feb. 2025.

[18] A. Wei *et al.*, "Agentic root cause analysis through evidence-grounded reasoning," arXiv:2607.22385, Jul. 2026.

[19] R. van den Brink and R. P. Gilles, "Axiomatizations of the conjunctive permission value for games with permission structures," *Games and Economic Behavior*, vol. 12, no. 1, pp. 113–126, 1996.

[20] R. B. Myerson, "Graphs and cooperation in games," *Mathematics of Operations Research*, vol. 2, no. 3, pp. 225–229, 1977.